Modül 21

Progressive Web Apps

Offline, Install, Push Notifications

Modül:	21 / 30
Ders Sayısı:	6 ders
Seviye:	Orta
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

🚀 Modül 21'e Hoş Geldin

PWA — web uygulamasını native app gibi kullanılabilir hale getirir. Offline çalışma, install edilebilirlik, push notification, background sync. Angular'ın service worker desteği ile kolay.

Öğrenecekler:

- ✓ @angular/pwa setup
- ✓ Service worker konfigürasyonu
- ✓ Manifest.json (install)
- ✓ Offline strategies (cache-first, network-first)
- ✓ Push notifications
- ✓ App update flow

DERS 21.1

PWA Setup

Angular projesine PWA ekleme.

⚙ Tek Komutla

```
ng add @angular/pwa

# Otomatik oluşur:
# - manifest.webmanifest
# - ngsw-config.json (service worker config)
# - src/assets/icons/ (icon'lar)
# - index.html güncellemesi
```

manifest.webmanifest

```
{
  "name": "Benim Uygulamam",
  "short_name": "Uygulamam",
  "theme_color": "#4f46e5",
  "background_color": "#ffffff",
  "display": "standalone",
  "scope": "./",
  "start_url": "./",
  "icons": [
    {
      "src": "assets/icons/icon-72x72.png",
      "sizes": "72x72",
      "type": "image/png"
    },
    {
      "src": "assets/icons/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ]
}
```

Build & Test

```
ng build

# Service worker sadece production build'de çalışır
# Test için:
npx http-server -p 8080 dist/my-app/browser

# Lighthouse PWA audit
# Chrome DevTools → Lighthouse → PWA
```

DERS 21.2

ngsw-config.json

Service worker davranışını kontrol et.

⚙ Asset Groups

```
{
  "$schema": "./node_modules/@angular/service-worker/config/schema.json",
  "index": "/index.html",
  "assetGroups": [
    {
      "name": "app",
      "installMode": "prefetch",
      "resources": {
        "files": [
          "/favicon.ico",
          "/index.html",
          "/manifest.webmanifest",
          "/*.css",
          "/*.js"
        ]
      }
    },
    {
      "name": "assets",
      "installMode": "lazy",
      "updateMode": "prefetch",
      "resources": {
        "files": [
          "/assets/**",
          "/*.(svg|cur|jpg|jpeg|png|webp|gif|otf|ttf|woff|woff2)"
        ]
      }
    }
  ]
}
```

Data Groups — API Cache

```
{
  "dataGroups": [
    {
      "name": "api-performance",
      "urls": ["/api/users/**", "/api/products/**"],
      "cacheConfig": {
        "strategy": "performance",   // Cache-first
        "maxSize": 100,
        "maxAge": "1h"
      }
    },
    {
      "name": "api-freshness",
      "urls": ["/api/notifications/**", "/api/messages/**"],
      "cacheConfig": {
        "strategy": "freshness",     // Network-first
        "maxSize": 50,
        "maxAge": "5m",
        "timeout": "3s"              // Network timeout, fallback to cache
      }
    }
  ]
}
```

DERS 21.3

Offline Strategies

Network olmadan çalışmak.

Online Status

```
@Injectable({ providedIn: 'root' })
export class NetworkService {
  isOnline = signal(navigator.onLine);

  constructor() {
    window.addEventListener('online', () => this.isOnline.set(true));
    window.addEventListener('offline', () => this.isOnline.set(false));
  }
}

@Component({
  template: `
    @if (!network.isOnline()) {
      <div class="offline-banner">⚠ Offline mod</div>
    }
  `
})
export class AppComponent {
  protected network = inject(NetworkService);
}
```

Offline Action Queue

```
@Injectable({ providedIn: 'root' })
export class OfflineQueueService {
  private queue = signal<QueuedAction[]>([]);
  private network = inject(NetworkService);

  constructor() {
    // Network geri geldiğinde queue'yu işle
    effect(() => {
      if (this.network.isOnline() && this.queue().length > 0) {
        this.processQueue();
      }
    });
  }

  add(action: QueuedAction) {
    if (this.network.isOnline()) {
      return this.execute(action);  // Direkt gönder
    }
    this.queue.update(q => [...q, action]);  // Queue'ya ekle
  }

  private async processQueue() {
    const pending = this.queue();
    for (const action of pending) {
      await this.execute(action);
    }
    this.queue.set([]);
  }
}
```

DERS 21.4

Cache Strategies

Cache-first vs Network-first vs Stale-while-revalidate.

4 Strateji

Strateji	Davranış	Use Case
Cache First	Cache'ten al, yoksa network	Statik içerik (CSS, font)
Network First	Network'ten al, başarısız→cache	Dinamik veri
Cache Only	Sadece cache	Offline-first
Network Only	Sadece network	Sensitif veri (auth)

Stale-While-Revalidate

En popüler strateji: Cache'ten anında göster, arka planda network'ten yenisi, cache'i güncelle.

```
// ngsw-config.json
{
  "name": "api",
  "urls": ["/api/**"],
  "cacheConfig": {
    "strategy": "performance",
    "maxAge": "1d",
    "maxSize": 100
  }
}

// Bu cache-first ile yakın davranır — Angular SW
// True SWR için custom service worker gerek
```

DERS 21.5

Push Notifications

Web push notification entegrasyonu.

Push API Kullanımı

```
import { SwPush } from '@angular/service-worker';

@Injectable({...})
export class NotificationService {
  private swPush = inject(SwPush);
  readonly VAPID_PUBLIC_KEY = 'YOUR_VAPID_KEY';

  async subscribe() {
    if (!this.swPush.isEnabled) {
      console.log('Push notifications not supported');
      return;
    }

    try {
      const subscription = await this.swPush.requestSubscription({
        serverPublicKey: this.VAPID_PUBLIC_KEY
      });

      // Backend'e subscription'ı kaydet
      await fetch('/api/subscribe', {
        method: 'POST',
        body: JSON.stringify(subscription),
        headers: { 'Content-Type': 'application/json' }
      });
    } catch (err) {
      console.error('Subscription failed', err);
    }
  }

  listenForMessages() {
    this.swPush.messages.subscribe(msg => {
      console.log('Push message:', msg);
    });

    this.swPush.notificationClicks.subscribe(({ notification, action }) => {
      window.focus();
      if (notification.data?.url) {
        window.location.href = notification.data.url;
      }
    });
  }
}
```

🖥️ Server-Side (Node.js)

```
const webpush = require('web-push');

webpush.setVapidDetails(
  'mailto:admin@example.com',
  VAPID_PUBLIC_KEY,
  VAPID_PRIVATE_KEY
);

// Send notification
webpush.sendNotification(subscription, JSON.stringify({
  notification: {
    title: 'Yeni mesaj',
    body: 'Birinden mesaj geldi',
    icon: 'assets/icons/icon-192x192.png',
    data: { url: '/messages/123' },
    actions: [
      { action: 'view', title: 'Görüntüle' },
      { action: 'dismiss', title: 'Kapat' }
    ]
  }
}));
```

DERS 21.6

App Update Flow

Yeni versiyon geldiğinde kullanıcıyı bilgilendirme.

SwUpdate Service

```
import { SwUpdate } from '@angular/service-worker';

@Injectable({...})
export class UpdateService {
  private swUpdate = inject(SwUpdate);

  constructor() {
    if (!this.swUpdate.isEnabled) return;

    // Yeni versiyon mevcut
    this.swUpdate.versionUpdates.subscribe(event => {
      if (event.type === 'VERSION_READY') {
        if (confirm('Yeni versiyon mevcut. Yenile?')) {
          window.location.reload();
        }
      }
    });

    // Periyodik kontrol (her 6 saat)
    setInterval(() => {
      this.swUpdate.checkForUpdate();
    }, 6 * 60 * 60 * 1000);
  }

  async forceUpdate() {
    try {
      const updated = await this.swUpdate.checkForUpdate();
      if (updated) {
        await this.swUpdate.activateUpdate();
        window.location.reload();
      }
    } catch (err) {
      console.error('Update failed', err);
    }
  }
}
```

UI Component

```
@Component({
  template: `
    @if (updateAvailable()) {
      <div class="update-banner">
        <p>Yeni versiyon mevcut!</p>
        <button (click)="refresh()">Yenile</button>
        <button (click)="dismiss()">Sonra</button>
      </div>
    }
  `
})
export class UpdateBannerComponent {
  private updateService = inject(UpdateService);
  updateAvailable = signal(false);

  constructor() {
    this.updateService.versionReady$.subscribe(() => {
      this.updateAvailable.set(true);
    });
  }

  refresh() { window.location.reload(); }
  dismiss() { this.updateAvailable.set(false); }
}
```

PWA Best Practices

- ✓ HTTPS zorunlu — service worker ancak HTTPS'de çalışır
- ✓ App shell pattern — header/footer cache, content lazy
- ✓ Update notification — kullanıcıya yeni versiyon bildir
- ✓ Offline fallback — /offline.html sayfası
- ✓ Icon variations — maskable, monochrome, vs.
- ✓ Lighthouse PWA score — 100/100 hedefle

Modül 21 Özeti

PWA artık temel beceri. Offline destekli, install edilebilir, push notification ile native app benzeri deneyim sunan uygulamalar yazabilirsin.

Neler Öğrendin?

- ✓ @angular/pwa kurulum
- ✓ Service worker config (asset groups, data groups)
- ✓ Offline strategies
- ✓ Cache stratejileri (4 yaklaşım)
- ✓ Push notifications
- ✓ App update flow

Sıradaki Modül

Modül 22 — Security. XSS, CSRF, Content Security Policy, sanitization.